MM811 Programming Assignment Report

Zhengyi Yan

Abstract

This report presents four experiments on MNIST and CIFAR-10: image classification, binary-latent reconstruction, autoregressive memorization, and latent image generation. The implementations were kept deliberately compact and were evaluated on fixed data splits. The **MNIST classifier** obtained **92.45% test accuracy**. The $3 \times 4 \times 4$ **binary autoencoder** exceeded **25 dB** on its **256-image training subset** but did not generalize to the independent test set. All three **autoregressive variants** met their memorization targets. For CIFAR-10, both **continuous and vector-quantized autoencoders** exceeded **25 dB**. The **absorbing-mask discrete latent diffusion model** obtained **FID 37.4657**, while the final **continuous latent Transformer with a per-token diffusion head** obtained **FID 28.8195**; both values use **10,000 generated images** and meet the required **FID** < 40. An independent seed reproduced the continuous result at **FID 28.8515**. The report separates reconstruction quality from generative quality and states every evaluation split explicitly.

1 Experimental protocol

MNIST images are scaled to $[0, 1]$. CIFAR-10 uses 45,000 training images, 5,000 validation images, and the official 10,000-image test set unless otherwise stated. The primary random seed is 42; Task 4.3 is independently repeated with seed 43. Models were trained with PyTorch on Apple MPS for early experiments and NVIDIA RTX 2080 SUPER GPUs for the longer CIFAR-10 runs.

Classification is measured by accuracy. Reconstruction uses per-image mean squared error (MSE) and Peak Signal-to-Noise Ratio

$$\mathrm{PSNR}(x, \hat{x}) = -10 \log_{10} \left(\mathrm{MSE}(x, \hat{x}) \right), \tag{1}$$

because the pixels lie in $[0, 1]$. The reported PSNR is the mean of the per-image values. Image generation is evaluated with Fréchet Inception Distance (FID) [4]; lower is better. The assignment targets are **accuracy** $> 90\%$, **PSNR** > 25 **dB**, the stated memorization rates, and **FID** < 40.

2 Task 1: MNIST image classification

2.1 Model and training

The classifier follows the compact classroom design: **four convolutions** reduce a 28×28 image to 64 features, followed by a **three-layer classification head**. **ReLU** is used after every hidden layer. The final layer returns ten logits, and the model is trained by cross-entropy. Only 1,024 training images and 256 validation images are used; model selection is based on validation accuracy.

Table 1: Task 1 network architecture.

Stage	Operation	Output size	Activation
Input	MNIST image	$N \times 1 \times 28 \times 28$	–
Features	Conv $1 \to 8$, 4×4, stride 2	$N \times 8 \times 14 \times 14$	ReLU
	Conv $8 \to 16$, 4×4, stride 2	$N \times 16 \times 7 \times 7$	ReLU
	Conv $16 \to 32$, 4×4, stride 2	$N \times 32 \times 3 \times 3$	ReLU
	Conv $32 \to 64$, 3×3	$N \times 64 \times 1 \times 1$	–
Classifier	Linear $64 \to 128 \to 256 \to 10$	$N \times 10$	ReLU/ReLU

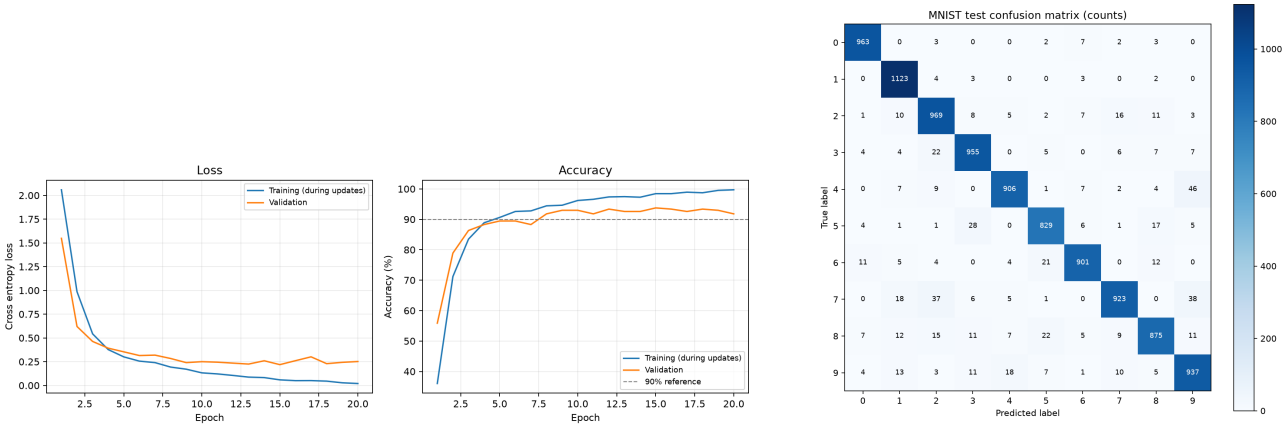
The core model is only a few lines:

```
self.feats = nn.Sequential(
    nn.Conv2d(1, 8, 4, 2, 1), nn.ReLU(),
    nn.Conv2d(8, 16, 4, 2, 1), nn.ReLU(),
    nn.Conv2d(16, 32, 4, 2, 1), nn.ReLU(),
    nn.Conv2d(32, 64, 3))
self.labs = nn.Sequential(
    nn.Linear(64, 128), nn.ReLU(),
    nn.Linear(128, 256), nn.ReLU(),
    nn.Linear(256, 10))
```

Adam with learning rate 10^{-3}, batch size 8, and 20 epochs was used. The best validation checkpoint occurred at epoch 18.

2.2 Results

The best validation accuracy was **91.41%**, and the corresponding accuracy on all **10,000 test images** was **92.45%**. Therefore Task 1 exceeds the required **90% threshold**.



(a) Training and validation curves from the detailed run.

(b) Test confusion matrix.

Figure 1: Task 1 diagnostics. The separate compact classroom implementation produced the reported **92.45% test accuracy**.

3 Task 2: binary latent representation

3.1 Architecture and binarization

The **encoder** maps an MNIST image to exactly $N \times 3 \times 4 \times 4 = 48$ values. During evaluation every latent value is strictly -1 or $+1$. A **straight-through estimator (STE)** makes the sign operation differentiable during back-propagation:

$$z = \operatorname{sign}(a), \qquad \frac{\partial z}{\partial a} \approx 1. \tag{2}$$

For the first five epochs, the hard sign is replaced by $2\sigma(10a) - 1$ to provide a smooth warm-up. The remainder of training uses the **hard STE**. The **decoder** mirrors the encoder with transposed convolutions and ends with a sigmoid.

Table 2: Task 2 autoencoder.

Part	Main operations	Output
Encoder	Conv $1 \to 64$, Conv $64 \to 128$ (stride 2), Conv $128 \to 256$ (stride 2), Linear	$N \times 3 \times 4 \times 4$
Binarizer	soft warm-up, then sign with STE	values in $\{-1, +1\}$
Decoder	Linear to $256 \times 7 \times 7$, two transposed convolutions, final Conv+sigmoid	$N \times 1 \times 28 \times 28$

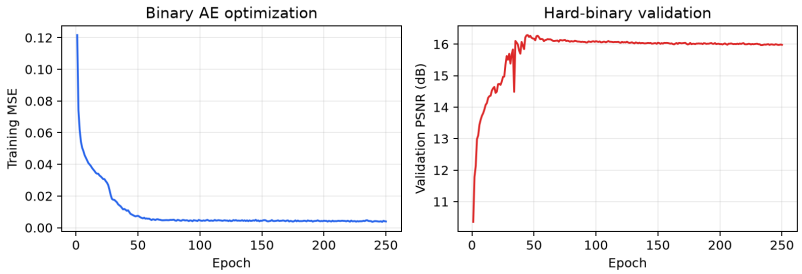
The training loop minimizes pixel MSE with Adam. A ReduceLROnPlateau scheduler monitors hard-binary validation MSE. The final ReLU experiment used 256 training images, 256 validation images, 250 epochs, and 3,121,201 parameters.

3.2 Results and interpretation

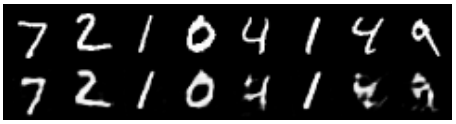
Table 3: Task 2 hard-binary results. Test evaluation uses all 10,000 official MNIST test images.

Checkpoint	Epoch	Train PSNR	Validation PSNR	Test PSNR
Best validation	45	**22.08**	**16.29**	**16.16**
Final epoch	250	**26.23**	**15.98**	**15.91**

The training subset exceeds **25 dB**, so the implementation demonstrates that a **48-bit bottleneck** can memorize the allowed **256-image subset**. The independent validation and test scores remain much lower. Thus the result is **Train only** when the assignment threshold is interpreted on the selected training subset, but it does not establish > 25 **dB generalization**. This distinction is important because the assignment does not specify the evaluation split.



(a) Training MSE and hard-binary validation PSNR.



(b) Test inputs (top) and reconstructions (bottom).

Figure 2: The widening and warm-up improve training reconstruction, while the validation curve exposes overfitting.

4 Task 3: autoregressive memorization

4.1 Problem definition and shared token representation

Task 3 contains three separate experiments: Sequence-to-Token, Sequence-to-Sequence, and Sequence-to-Histogram. All three model the same autoregressive factorization,

$$p(Z) = \prod_{i=1}^{16} p(z_i \mid z_{<i}), \tag{3}$$

but differ in how the prefix is presented and how the next-token distribution is represented. The first two experiments use 100 images, as allowed by the assignment. The histogram experiment uses 1,024 images, exceeding its 512-image requirement.

The frozen Task 2 encoder first converts each MNIST image into a strict signed-binary tensor $z \in \{-1, +1\}^{3\times4\times4}$. At each of the 4×4 spatial locations, the three channels are converted losslessly to one decimal token:

$$t = 4b_0 + 2b_1 + b_2, \qquad b_c = \mathbb{1}[z_c > 0]. \tag{4}$$

For example, channel bits $(1, 0, 1)$ become token 5. Each image is therefore a length-16 sequence over $\{0, \ldots, 7\}$. The inverse bit shifts recover the original 48 signed bits exactly. Two additional symbols are used internally: `BOS`=8 marks the beginning of a sequence and `PAD`=9 pads prefixes, giving an input vocabulary of size ten. The predicted output vocabulary remains the eight valid image tokens.

```
bits = (latent > 0).long()
tokens = bits[:, 0] * 4 + bits[:, 1] * 2 + bits[:, 2]
tokens = tokens.flatten(1)              # [N, 16], values 0,...,7
```

For evaluation, G denotes the 1,000 generated sequences and S_{train} is the set of complete training sequences. A sample counts as memorized only if all 16 tokens match one training sequence exactly:

$$R_{\text{mem}} = \frac{1}{|G|} \sum_{g \in G} \mathbb{1}[g \in S_{\text{train}}]. \tag{5}$$

This is stricter than comparing individual token accuracy or visual similarity. All neural sampling uses temperature 0.2. A low temperature is appropriate because the task explicitly measures memorization rather than sample diversity.

4.2 Task 3.1: Sequence-to-Token

4.2.1 Input, output, and architecture

One length-16 training sequence creates 16 supervised prefix examples. At position i, the input is $[\text{BOS}, z_0, \ldots, z_{i-1}]$, padded to length 16, and the target is the single next token z_i. Thus 100 images produce 1,600 prefix-to-token training pairs.

The model embeds each symbol into 64 dimensions and passes the prefix through a **one-layer GRU with hidden size 128**. The hidden state at the final non-padding position is projected to **eight logits**. This construction supports variable prefix lengths while producing exactly one categorical distribution.

Table 4: Sequence-to-Token architecture.

Stage	Operation	Output
Input	Padded prefix with BOS and PAD symbols	$N \times 16$
Embedding	Vocabulary $10 \to 64$	$N \times 16 \times 64$
Recurrent model	One-layer GRU, hidden size 128	$N \times 16 \times 128$
Prefix selection	Select the last valid hidden state	$N \times 128$
Classifier	Linear $128 \to 8$	$N \times 8$ logits

4.2.2 Training and generation

The model is trained for 400 epochs with Adam, learning rate 2×10^{-3}, batch size 128, and cross-entropy loss

$$\mathcal{L}_{\text{S2T}} = -\log p_\theta(z_i \mid z_{<i}). \tag{6}$$

The mean training loss decreases from 2.0920 to 0.2934. Generation begins with BOS. The GRU predicts one token, the sampled token is appended to the prefix, and the procedure repeats until 16 tokens have been produced.

Among **1,000 generated sequences**, all **1,000** exactly match a training sequence. The generated set contains **70 unique sequences**. Therefore the memorization rate is **100.00%**, exceeding the required **80%**.

4.3 Task 3.2: Sequence-to-Sequence

4.3.1 Input, output, and causal masking

The complete target is the original sequence $(z_0, \ldots, z_{15})$. The input is shifted right by one position:

$$x = (\text{BOS}, z_0, \ldots, z_{14}), \qquad y = (z_0, z_1, \ldots, z_{15}). \tag{7}$$

A **causal attention mask** prevents output position i from reading z_i or future targets. Consequently, all 16 next-token distributions are trained in one forward pass without violating the autoregressive factorization.

Table 5: Sequence-to-Sequence architecture.

Stage	Operation	Output
Input	Right-shifted 16-token sequence	$N \times 16$
Token embedding	Vocabulary $10 \to 128$	$N \times 16 \times 128$
Position embedding	16 learned 128-D vectors, added to tokens	$N \times 16 \times 128$
Transformer	Two causal encoder blocks, four heads, FFN width 512	$N \times 16 \times 128$
Output head	LayerNorm and Linear $128 \to 8$	$N \times 16 \times 8$ logits

4.3.2 Training and generation

The loss sums the token-level cross-entropies across all positions:

$$\mathcal{L}_{\mathrm{S2S}} = -\frac{1}{16} \sum_{i=0}^{15} \log p_\theta(z_i \mid z_{<i}). \tag{8}$$

Adam with learning rate 2×10^{-3} is used for 400 epochs. The training loss decreases from 2.2268 to 0.2884. Although training predicts all positions in parallel, generation remains autoregressive: after every forward pass, only the final next-token distribution is sampled and appended to the input.

All **1,000 generated sequences** exactly match the training set, with **71 unique generated sequences**. The resulting **100.00%** memorization rate exceeds the required **80%**.

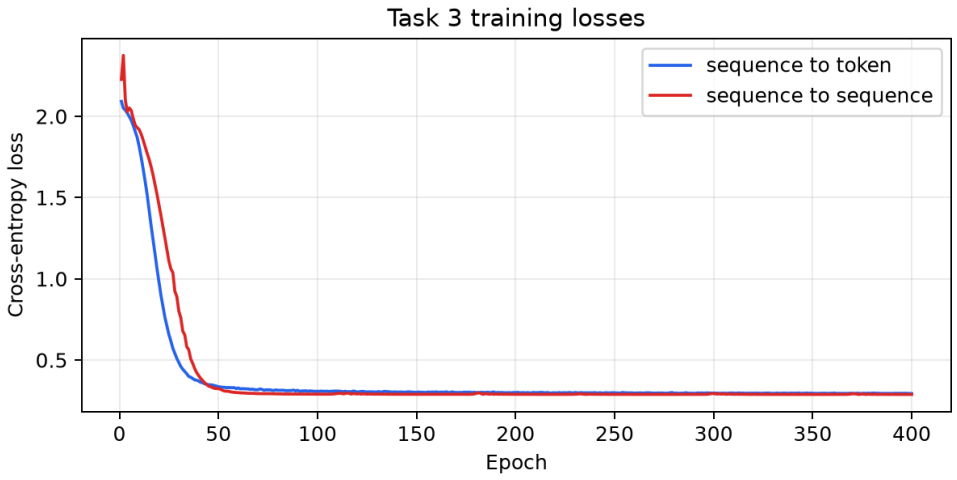


Figure 3: Cross-entropy histories for the Sequence-to-Token GRU and causal Sequence-to-Sequence Transformer. Both converge rapidly on the 100-image memorization set.

4.4 Task 3.3: Sequence-to-Histogram

4.4.1 Histogram targets and neural predictor

For every observed prefix $u = z_{<i}$, the training data first define an eight-bin count vector:

$$h_u(k) = \sum_{Z \in \mathcal{D}} \mathbb{1}[z_{<i} = u] \mathbb{1}[z_i = k], \qquad k \in \{0, \ldots, 7\}. \tag{9}$$

The normalized next-token distribution is

$$p(z_i = k \mid z_{<i} = u) = \frac{h_u(k)}{\sum_{j=0}^{7} h_u(j)}. \tag{10}$$

The model processes 1,024 images, producing 16,384 prefix observations. Prefixes that are identical are merged and their normalized histograms become soft training targets. The predictor has the same compact recurrent form as the Sequence-to-Token model: a **64-dimensional embedding**, a **one-layer GRU with hidden size 256**, and a **linear** $256 \to 8$ **output**. Applying softmax to its eight logits yields the predicted next-token histogram. This satisfies the requirement that the input to the network is the prior sequence and its output is a probability histogram.

```
target = counts.float() / counts.sum()
log_prob = log_softmax(model(prefix, prefix_length), dim=1)
loss = -(target * log_prob).sum(dim=1).mean()
```

4.4.2 Generation and result

Generation starts from BOS. At every position, the GRU predicts a new eight-bin histogram from the generated prefix and samples the next token; the training dictionary is not consulted during generation. Adam with learning rate 2×10^{-3} is used for 50 epochs, reducing the soft-label cross-entropy from 2.0818 to 0.0640.

The training data contain **1,023 unique sequences**. The neural model produces **35 unique sequences** among **1,000 samples**, of which **999** exactly occur in the training set. Its memorization rate is therefore **99.90%**, exceeding the required **99%** while using more than the required **512 images**.

4.5 Comparison of the three Task 3 experiments

Table 6: Task 3 memorization results from 1,000 generated sequences per method.

Model	Train images	Unique train	Unique generated	Match rate	Target
Sequence-to-Token	**100**	**100**	**70**	**100.00%**	$> 80\%$ **Pass**
Sequence-to-Sequence	**100**	**100**	**71**	**100.00%**	$> 80\%$ **Pass**
Sequence-to-Histogram	**1,024**	**1,023**	**35**	**99.90%**	$> 99\%$ **Pass**

All three approaches satisfy their respective thresholds. The first GRU and the Transformer learn hard next-token targets from the same 100 sequences, whereas the histogram GRU learns soft empirical distributions from 1,024 sequences. Generation is autoregressive for all three methods and never retrieves a completed sequence from the training set.



(a) Sequence-to-Token **(b)** Sequence-to-Sequence **(c)** Sequence-to-Histogram

Figure 4: Images obtained by decoding generated binary token sequences. The metric is exact sequence membership rather than visual similarity.

5 Task 4: CIFAR-10 latent image generation

5.1 Task 4.1: high-performance autoencoders

The **continuous autoencoder** produces $N \times 5 \times 8 \times 8$ latents. The discrete model is a **VQ-VAE** [1]: its encoder produces a 10-dimensional vector at each of 8×8 positions, and every vector is replaced by the nearest entry in a **512-vector codebook**. Thus the representation used by the generator is an 8×8 grid of integer indices in $[0, 511]$, while the quantized embedding tensor is $N \times 10 \times 8 \times 8$.

The VQ objective is

$$\mathcal{L} = \|x - \hat{x}\|_2^2 + \|\,\mathrm{sg}[z_e] - e\|_2^2 + \beta\|z_e - \mathrm{sg}[e]\|_2^2, \tag{11}$$

where e is the selected codebook vector and sg stops gradients. Residual blocks are used inside both encoder and decoder.

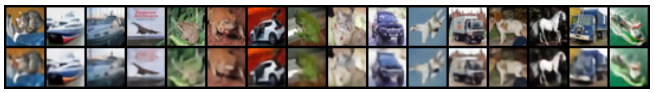
Table 7: Task 4.1 CIFAR-10 reconstruction results.

Model	Latent	Train PSNR	Val. PSNR	Test PSNR	Status
Continuous AE	$5 \times 8 \times 8$	**30.99**	**30.25**	**30.30**	**Pass**
Perceptual/noisy continuous AE	$5 \times 8 \times 8$	–	**29.05**	**29.09**	**Pass**
VQ-VAE	8×8 indices, embeddings $10 \times 8 \times 8$	–	–	**25.24**	**Pass**

The **perceptual/noisy continuous AE** is fine-tuned for Task 4.3 with **pixel MSE, LPIPS weight 0.08, latent L_2 weight 10^{-5}, and Gaussian latent noise** at 0.1 times the channel standard deviation. It obtains **test PSNR 29.0898 dB**, **test LPIPS 0.0600**, and **reconstruction FID 11.6267**. The **VQ-VAE** used for Task 4.2 uses **all 512 codes** on the test set and has **codebook perplexity 489.1**, indicating that the codebook did not collapse. Its **reconstruction FID is 34.6748**; reconstruction FID is a useful decoder-quality diagnostic but is not a generative FID.



(a) Continuous AE reconstructions.



(b) VQ-VAE reconstructions.

Figure 5: CIFAR-10 inputs and reconstructions.

5.2 Task 4.2: discrete latent diffusion

The generator operates directly on the **VQ-VAE's 8×8 grid of integer indices**. A special **[MASK] token** defines an **absorbing discrete diffusion process with 64 noise levels**. At each training step, the selected time determines the masking ratio, and cross-entropy is evaluated only at corrupted positions. The denoiser is a **17.80-million-parameter adaptive pre-layer-normalized Transformer** with width 320, ten blocks, eight attention heads, no dropout, exponential-moving-average weights, and time and class conditioning. Labels are randomly dropped during training to support **classifier-free guidance**.

Sampling starts from 64 [MASK] tokens and iteratively predicts and commits high-confidence codebook indices according to a **cosine unmasking schedule**, after which the frozen VQ-VAE decoder converts the indices to RGB images. The final configuration uses 12 refinement steps, linearly increasing classifier-free guidance to 5.0, token temperature 1.1, and confidence-choice temperature 4.5. **Horizontal flipping is applied to RGB images before VQ encoding**; this is important because flipping a flattened latent sequence would not implement the same spatial augmentation.

Table 8: Final Task 4.2 generation evaluations using standard `pytorch-fid` 0.3.0 with 2,048-dimensional Inception features.

Training configuration	Images	Final masked CE	FID	Target
200 epochs, no augmentation	**10,000**	**5.136510**	**45.1325**	**Not yet**
Legacy 7.13M denoiser, 230 epochs, corrected RGB flip	**10,000**	**5.380221**	**40.9662**	**Not yet**
200 epochs, RGB horizontal flip	**10,000**	**5.296559**	**37.4657**	**Pass**

All values compare **10,000 generated images** with the official **10,000-image CIFAR-10 test set** using the same standard FID implementation. Correcting the horizontal-flip procedure and extending the earlier 7.13M-parameter denoiser from 200 to 230 epochs produced **FID 40.9662**, essentially unchanged from its **40.9438 baseline**. The final **17.80M-parameter adaptive model with RGB horizontal flipping** achieves **FID 37.4657**, beating the required threshold by **2.5343 points**. The no-augmentation ablation has a lower final cross-entropy but a worse FID, demonstrating that masked-token likelihood is only a training diagnostic and cannot replace the required image-space metric.

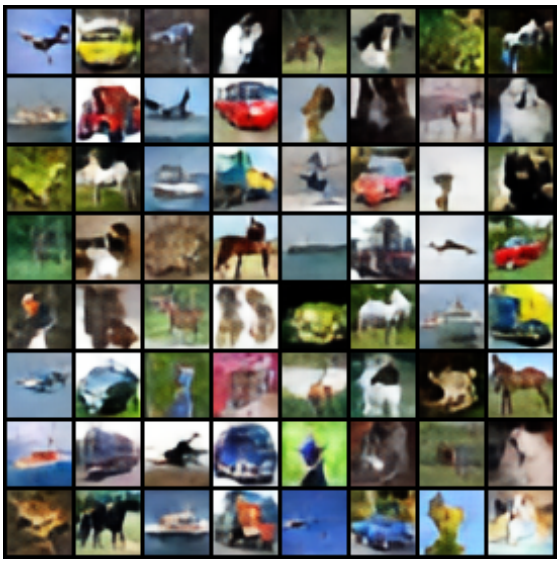


Figure 6: Representative samples from a discrete latent diffusion development checkpoint. The final augmented run was evaluated separately on **10,000 generated images** and obtained **FID 37.4657**.

5.3 Task 4.3: continuous latent Transformer

The **perceptual/noisy continuous autoencoder** maps every image to an 8×8 grid of five-dimensional real vectors. Clean and horizontally flipped RGB training images are encoded separately, producing 100,000 sequences of length 64. Each latent channel is standardized using training-set statistics. In accordance with the provided architecture, each cell is projected by $\mathrm{Linear}(5, 512)$ and processed by a **causal, pre-layer-normalized Transformer** with eight blocks, eight attention heads, feed-forward width 2,048, dropout 0.1, learned positional embeddings, and a learned beginning-of-sequence vector.

An explicit **16-component diagonal Gaussian-mixture output** initially reached only **FID 53.9985**. The final model instead uses each causal state h_i to condition a **per-token diffusion head** for the next five-dimensional cell, following the continuous-token motivation of GIVT [3]. The head contains **four width-512 adaptive-layer-normalized MLP blocks and a 1,000-step cosine noise schedule**. For clean cell z_i, noise ϵ, and cumulative signal level $\bar{\alpha}_t$, training uses

$$z_{i,t} = \sqrt{\bar{\alpha}_t}\, z_i + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \tag{12}$$

$$\mathcal{L}_{\text{diff}} = \|\epsilon - \epsilon_\theta(z_{i,t}, t, h_i)\|_2^2 . \tag{13}$$

Four noise draws are used per token and **exponential-moving-average weights** with decay 0.999 are retained. The complete **Transformer and diffusion head** contain **37,995,013 trainable parameters**.

Generation remains **autoregressive across the 64 raster-ordered cells**. For each cell, the **diffusion head** performs 50 stochastic denoising steps with $\eta = 1$, temperature 1.0, and normalized-latent clipping at 6.0; the frozen decoder then maps the completed $5 \times 8 \times 8$ latent to RGB. Both runs use 50 epochs and select the EMA checkpoint with the lowest validation noise MSE. Seed 42 selects **epoch 16 at 0.408118**, while seed 43 selects **epoch 16 at 0.407760**.

Table 9: Task 4.3 standard generation evaluations. Every FID uses 10,000 generated images against the official 10,000-image CIFAR-10 test set.

Output distribution	Seed	Epochs	Parameters	FID	Target
16-component diagonal GMM	42	200	**6.38M**	**53.9985**	**Not yet**
Per-token diffusion head	42	50	**38.00M**	**28.8515**	**Pass**
Per-token diffusion head	43	50	**38.00M**	**28.8195**	**Pass**

The two independent final FIDs differ by only **0.0320**, providing evidence that the improvement is reproducible rather than a favorable single seed. The better seed-43 result beats the assignment threshold by **11.1805 FID points**. The result also shows why likelihood-style diagnostics alone are insufficient: the decoder's perceptual quality and tolerance to imperfect latent samples were important to image-space FID.

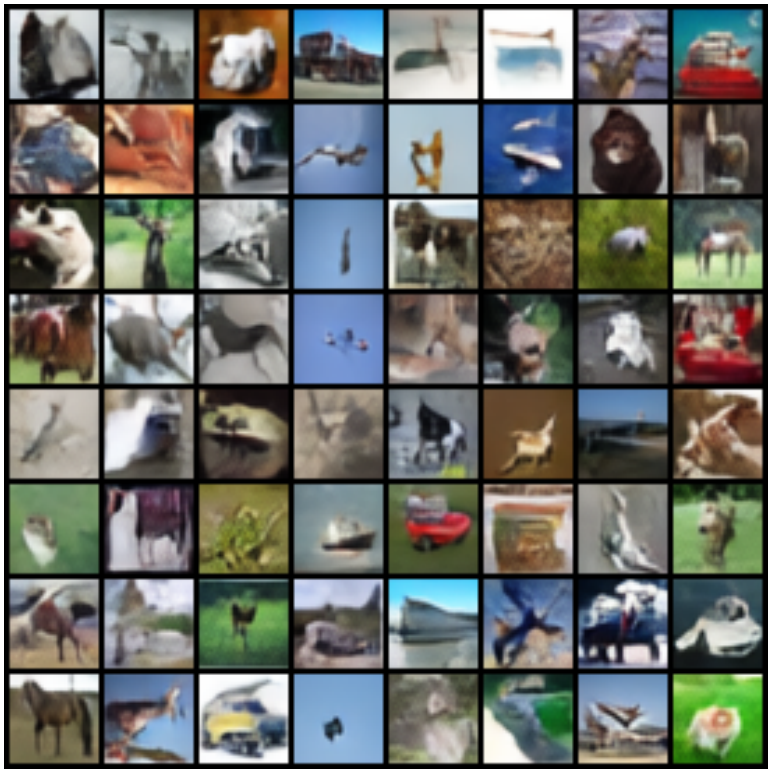


Figure 7: Samples from the seed-43 continuous latent Transformer with a per-token diffusion head. The standard **10,000-image evaluation** obtained **FID 28.8195**; seed 42 independently obtained **FID 28.8515**.

6 Summary and limitations

Table 10: Comparison with the assignment thresholds.

Task	Best verified result	Status
1	**MNIST test accuracy 92.45%** ($> 90\%$)	**Pass**
2	**256-image train PSNR 26.23 dB; test PSNR 15.91 dB**	**Train only**
3	**100%, 100%, and 99.90% exact memorization**	**Pass**
4.1 continuous	**Best test PSNR 30.30 dB; generator AE 29.09 dB; latent** $5 \times 8 \times 8$	**Pass**
4.1 discrete	**Test PSNR 25.24 dB, latent** 8×8 **indices with 10-D code vectors**	**Pass**
4.2	**FID 37.4657 (10,000 images); target** < 40	**Pass**
4.3	**FID 28.8195 (seed 43) and 28.8515 (seed 42), 10,000 images**	**Pass**

The experiments demonstrate the required architectures and leave Task 2 generalization through an extreme **48-bit code** as the principal limitation. Task 4.2 meets the generation target through **absorbing-mask diffusion, spatially correct augmentation, EMA, and iterative guidance**. Task 4.3 also meets the target: **perceptual/noisy autoencoder training** lowers the decoder's reconstruction floor, while a **conditional per-token diffusion head** models the multimodal distribution of each continuous latent cell more effectively than an explicit Gaussian mixture. Independent seeds reproduce essentially the same FID. Throughout the report, training loss, reconstruction metrics, and generative FID remain separate because only the required image-space evaluation establishes generation quality.

Reproducibility

Each stage is implemented in a separate, compact source file:

Task	Source file
1	`task1_classroom.py`
2	`task2_optimized.py`
3	`task3_autoregressive.py`
4.1	`task4_autoencoders.py`, `task4_vqvae_masked.py`
4.2	`task4_vq_diffusion_final.py`
4.3	`task4_continuous_ae_perceptual.py`, `task4_continuous_token_diffusion.py`
Evaluation	`task4_fid_standard.py`

Representative commands are:

```
python task1_classroom.py
python task2_optimized.py --activation relu
python task3_autoregressive.py --generate 1000
python task4_autoencoders.py --epochs 100
python task4_vqvae_masked.py --epochs 150
python task4_vq_diffusion_final.py --vqvae <vqvae.pt> --epochs 200 --augmentation pixel-flip --
    generate 10000
python task4_continuous_ae_perceptual.py --init-ae <continuous.pt> --epochs 30 --lpips-weight
    0.08 --latent-noise 0.1
python task4_continuous_token_diffusion.py --autoencoder <autoencoder.pt> --epochs 50 --
    augmentation pixel-flip --sample-steps 50 --generate 10000
python task4_fid_standard.py <generated_images.pt> --images 10000
```

References

[1] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural Discrete Representation Learning," *Advances in Neural Information Processing Systems*, 2017.

[2] H. Chang, H. Zhang, L. Jiang, C. Liu, and W. T. Freeman, "MaskGIT: Masked Generative Image Transformer," *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[3] M. Tschannen, C. Eastwood, and F. Mentzer, "GIVT: Generative Infinite-Vocabulary Transformers," *European Conference on Computer Vision*, 2024.

[4] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," *Advances in Neural Information Processing Systems*, 2017.